A Study of SIEM Rule Tuning Techniques for Reducing Alert Fatigue and False Positive Alerts Across Different Attack Scenarios in SOC Environments

Oluwadarasimi Sulaiman · MSc Cybersecurity, University of Hertfordshire · Portfolio case study

For my MSc Cybersecurity dissertation at the University of Hertfordshire, I built a small SOC lab in Splunk, attacked it three different ways, and tuned the detection rules by hand to cut out false alarms without missing the real attacks. The project was graded 80% and contributed to an overall Distinction.

The headline: for SSH brute force, benign alerts dropped from 41 to 2, about 95%. For privilege escalation, 11 sudo alerts dropped to the 1 that mattered. For port scanning, about 4,852 firewall events became 1 clearly identified scanner. Every real attack was still caught.

The bigger finding was that each attack needed a different kind of fix. There is no single tuning trick that works everywhere.

The problem

SIEM tools flag anything that looks suspicious, and most of what they flag turns out to be harmless. Aktar et al. (2024) report that over 90% of SIEM alerts have no link to a real incident. Tariq et al. (2025) found analysts spend more than a quarter of their time on false positives.

When that happens every day, analysts start assuming the next alert is probably nothing too. That is alert fatigue, and it is exactly when a real attack slips through.

Most research answers this with machine learning or automated tools. Those need data and infrastructure that small organisations simply do not have. Manual rule tuning is what analysts actually do day to day, but it has been tested far less. I wanted to measure how well it really works, and whether it works the same way for different kinds of attack.

The lab I built

Everything ran in an isolated virtual network in VMware Workstation on my own laptop, so no real systems were touched.

Part	What I used	Its job
Target	Ubuntu at 192.168.80.130	The “company” server being watched
SIEM	Splunk Enterprise 9.4.1 on the Ubuntu machine	Collects auth.log, syslog and firewall logs and runs my searches
Attacker	Kali Linux at 192.168.80.128	Runs Hydra, the privilege escalation and Nmap
Users	Seven employee accounts plus an admin account	Generate normal, harmless activity to test the rules against

I chose Splunk because it is widely used in real SOCs and its search language (SPL) made it easy to change one thing in a rule and measure the effect. I modelled a micro business with seven staff, which is the size of organisation that cannot afford enterprise detection tools.

For every attack I kept two separate time windows: a benign window with only normal staff activity, and an attack window. A good rule should be quiet in the first and still catch the attack in the second.

How the project really went

The final results look neat, but getting there involved a lot of going back and fixing my own method. These were the turning points.

I changed my third attack. My original plan included phishing. My supervisor pointed out it could not be simulated meaningfully in a two machine lab, so I replaced it with privilege escalation.

I stopped trusting untimed numbers. When I first ran the attacks, I did not record exact start and end times. My first Hydra count came back as 5,442 events, but I could not prove which of those belonged to the attack and which were background noise. I re-ran the attacks with exact timestamps, and from then on I only used numbers I could pin to a precise window in Splunk.

I caught myself testing the wrong attack. My first attempt at privilege escalation used an automated SSH login module. When I looked closely, I realised it was really another password guessing attack, not privilege escalation at all. Rather than quietly rename it, I built a genuine escalation: I planted a sudo misconfiguration and exploited it to go from a normal user to root.

I built a proper benign dataset. My first set of normal activity was too thin to test anything. I gave each of the seven employees a behaviour profile (one who always logs in cleanly, one who often mistypes, one who types usernames wrong) and kept a separate ground truth log of what each person did and when. That way every alert could later be marked as a true or false positive.

I double checked numbers that did not make sense. At one point my per user counts contradicted earlier results. It turned out a table of successful logins had been read as failed logins. Once I separated the two, the data was consistent, and I froze the dataset so it could not change while I tuned.

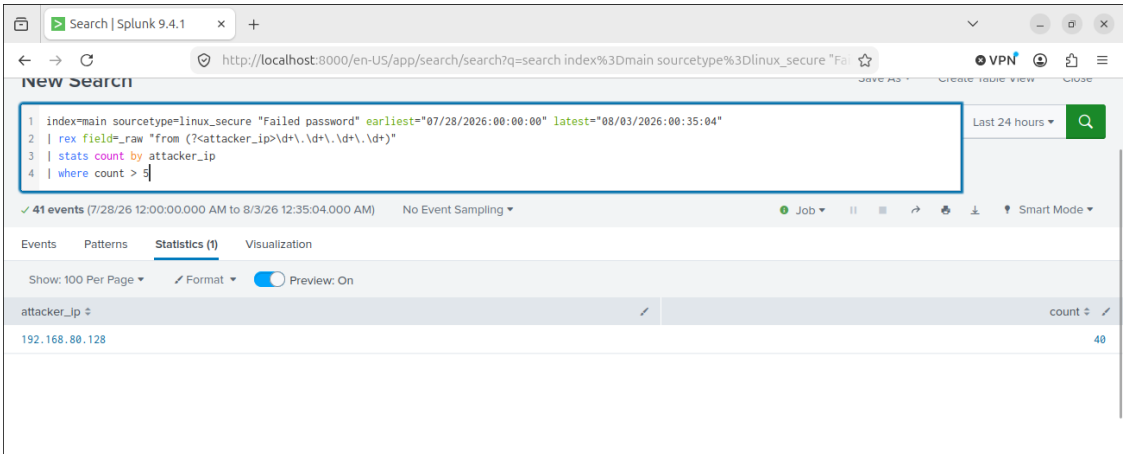
I changed one thing at a time. Each rule version changed a single piece of logic and was tested against both the benign window and the attack window, so I could see exactly what each change did.

Attack 1: SSH brute force, the volume problem

I ran Hydra from Kali against the root account for ten minutes, which produced 310 failed logins. Separately, my seven employees produced 41 failed logins over a week of normal use, just from typos.

Version	What the rule did	Benign window	Attack caught
V0	Alert on every failed login	41 alerts	Yes
V1	Group by source IP, flag more than 5	1 alert, but useless	Yes
V2	Group by target user, flag more than 5	2 accounts flagged	Yes, root with 310

V1 looked like a win on paper, but it failed in practice. All my employees and the attacker connected from the same machine, so grouping by IP lumped everyone together into one alert with no way to tell who was doing what. In a real office, staff behind one VPN or shared network would cause the same problem.

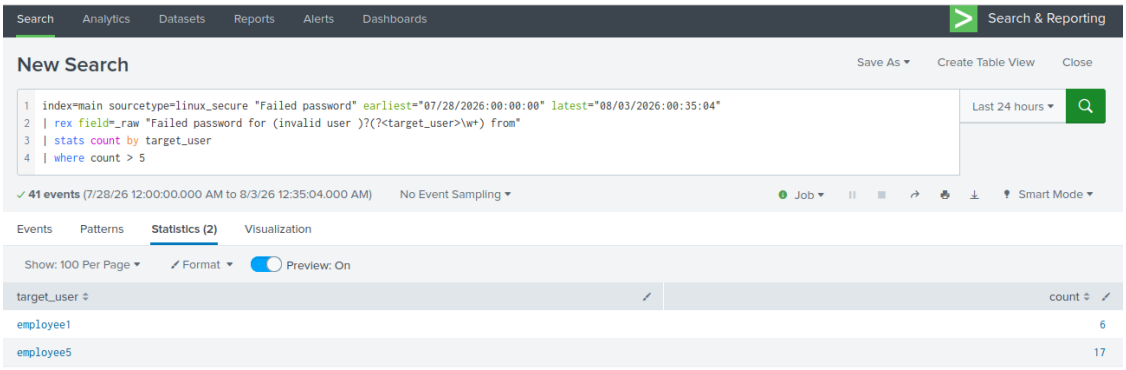


Before: grouping by source IP gave one alert for the shared address 192.168.80.128, with no way to tell which employee was which.

Switching to group by the account being targeted fixed it. Only employee5 (17 failures) and employee1 (6) were flagged, and root still stood out with 310.

The threshold needed care too. Two employees had exactly 5 failures. Flagging more than 5 left them out; flagging 5 or more would have doubled the false positives from 2 to 4. I set the threshold from what my own baseline showed, not from a number I picked in advance.

```
index=main sourcetype=linux_secure "Failed password"
| rex field=_raw "Failed password for (invalid user
)?(?<target_user>\w+) from"
| stats count by target_user
| where count > 5
```



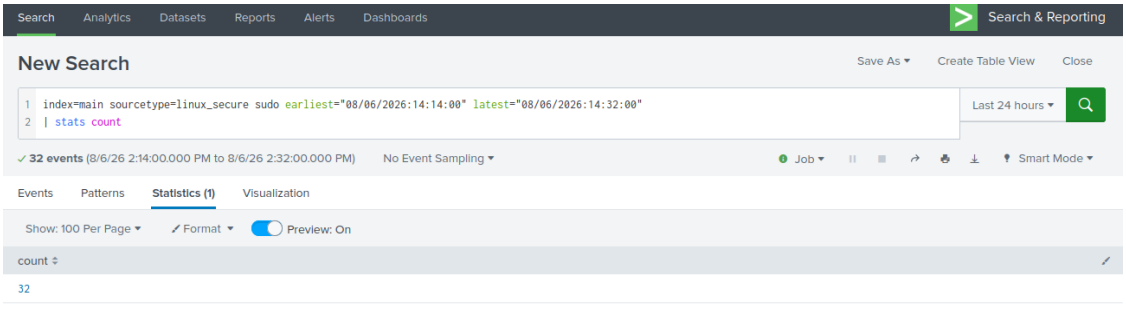
The tuned brute force rule in Splunk over the benign week: 41 failed logins in, only 2 accounts flagged.

Attack 2: privilege escalation, the context problem

I gave employee1 a deliberately bad sudo rule that let them run find as root without a password. From Kali, I logged in as employee1, spotted the rule with sudo -l, and used find to open a root shell (a known technique, MITRE ATT&CK T1548.003). In the same window, the admin account dara ran 10 normal sudo commands such as updates and service restarts.

Stage	What the rule did	Results	Attacker visible
Raw search	Every line mentioning sudo	32	Yes, buried
Refined	Only real commands (COMMAND field present)	11	Yes, among 10 admin commands
Tuned	Commands run as root, excluding the authorised admin	1	Yes, on its own

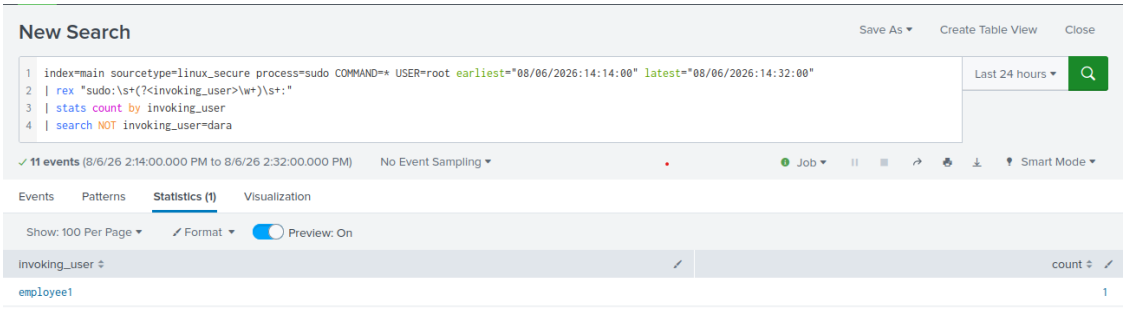
The first surprise was that 21 of the 32 raw results were not people at all. They were automatic PAM session messages the system writes around every sudo command. Without reading the logs properly, I would have counted them as activity.



Before: a raw search for sudo returned 32 lines, 21 of them machine generated PAM noise.

The bigger point is that the attack looked exactly like a normal admin command. It happened once, so there was no volume to set a threshold on. The only difference was who ran it. The fix was to tell the rule who is allowed to act as root.

In my lab I excluded dara by name. In a real SOC this would be a list of approved admins, so the rule stays correct as staff change.



The tuned privilege escalation rule: 11 sudo commands in the window, only employee1's escalation left.

Attack 3: port scanning, the precision problem

This attack needed a completely different way of judging success. For brute force and privilege escalation, I measured how many false alarms disappeared. For port scanning, that measure does not make sense, so I used different criteria. This section explains why.

A new log source

A port scan does not try to log in, so it leaves nothing in auth.log, the log I used for the first two attacks. I needed the firewall's view instead.

I turned on the Ubuntu firewall (UFW), raised its logging to high, and added /var/log/ufw.log to Splunk as its own source type, ufw. Splunk then pulled out the fields I needed from every firewall line: SRC (which machine sent the traffic), DPT (which port it tried to reach) and PROTO (the protocol).

The normal baseline

First I recorded a short window of normal activity on 12 August, from 17:23 to 17:26. The admin dara logged in over SSH three times and employee1 once.

I then counted how many different ports each machine touched. The highest was 4, from Ubuntu itself talking to normal services such as DNS (port 53), web (80 and 443) and time (123). Kali, logging in normally, touched only 1 port, which was SSH on 22.

One honest limitation: SSH to an allowed port does not create blocked firewall events, so my normal window was mostly background traffic rather than busy staff activity. It still gave me a clear ceiling for what normal looks like: 4 ports.

The first scan that did not work

My first scan covered ports 1 to 1,000 and finished in about three seconds. Splunk only showed 10 ports.

My first thought was that Splunk was losing data. To check, I searched the raw firewall log file on Ubuntu directly. It also only had 10 ports, so Splunk was fine. The real cause was that the scan was so fast the firewall's logging was rate limited and most probes were never written down.

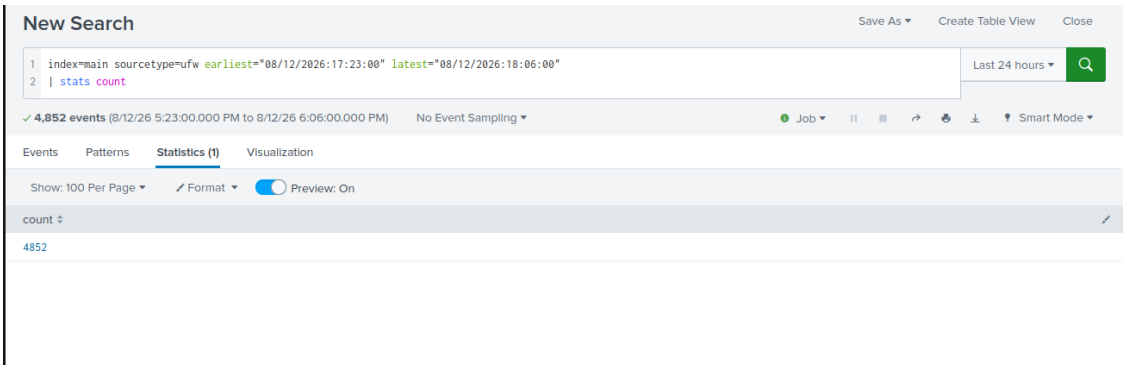
I fixed it in two ways: logging set to high, and a 20 millisecond pause between probes.

```
nmap -Pn -p 1-1000 --scan-delay 20ms 192.168.80.130
```

The definitive scan ran from 18:04:48 to 18:05:55, and the log captured 998 different ports from Kali. I recorded this as a deliberate change to my method. Without it, I would have been testing the firewall's logging limits instead of my detection rule. It is also why I chose Nmap over Masscan, which was too fast to capture reliably.

Why I did not use a percentage

The untuned search simply counted every firewall event in the whole window. It returned about 4,852 lines, a flood with no clear sign of who was scanning.



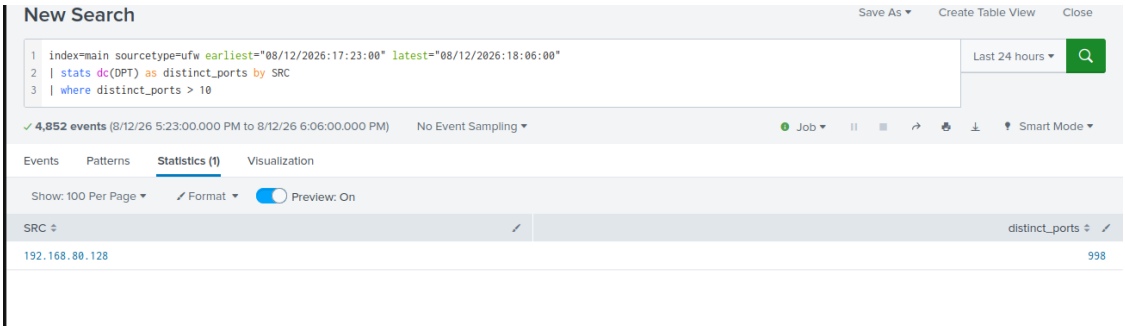
Before: counting every firewall event returned 4,852 lines and no sign of who was scanning.

The tuned rule returned 1 result. But it is not counting the same thing. The first number counts events and the second counts machines. Calling it a 99.98% reduction would be comparing apples with oranges, so I judged this rule on precision instead.

Criterion	What I checked	Result
Scanner identified	Does the rule name the attacking machine?	Yes, Kali at 192.168.80.128
Nothing else flagged	Are any normal machines caught?	No, Kali was the only result
Clear gap	How far apart are normal and attack?	4 ports normal against 998 in the scan
Threshold justified	Is the cut off based on my own data?	Yes, 10 sits above the normal ceiling of 4 and far below the scan

The tuned rule, line by line

```
index=main sourcetype=ufw
| stats dc(DPT) as distinct_ports by SRC
| where distinct_ports > 10
```



The tuned scanning rule: 4,852 firewall events in, one source out, Kali with 998 distinct ports.

The first line looks only at firewall logs. The second line counts, for each sending machine, how many different ports it tried, rather than how many lines it produced. The third line keeps only machines that tried more than 10 different ports.

The key idea is changing what the rule counts. One firewall line on its own means nothing. What makes a scan a scan is one machine knocking on lots of different doors. Counting distinct ports per machine captures exactly that, which matches how MITRE ATT&CK describes network service discovery (T1046).

What this rule would miss

A careful attacker could scan only a handful of ports, stay under 10, or spread the scan across several machines. My lab did not test those, so they are a fair next step.

Results at a glance

Attack	Kind of problem	Before tuning	After tuning	Real attack still caught
SSH brute force	Volume	41 benign alerts	2 accounts flagged	Yes, root with 310 failures
Privilege escalation	Context	11 sudo commands (32 raw lines)	1	Yes, employee1
Port scanning	Precision	About 4,852 firewall events	1 scanning source	Yes, Kali

Each attack needed a different fix: counting by user for brute force, knowing who is authorised for privilege escalation, and counting distinct ports for scanning.

What I learned and what I would test next

Good tuning starts with understanding the attack, not with picking a threshold. Brute force, privilege escalation and scanning each leave a different fingerprint in the logs, and the rule has to be built around that fingerprint.

A baseline of normal activity is part of the job, not a warm up. I could only justify my brute force threshold after I had seen how my employees actually behaved.

Check the logs before you judge the rule. Both the rate limited firewall log and the PAM noise would have given me wrong conclusions if I had trusted the raw numbers.

The limits are real. My lab had seven users, scripted normal activity and searches I ran by hand rather than scheduled alerts. I also measured alert volume, not how much time it actually saves an analyst.

If I took this further, I would test the brute force rule against password spraying, where an attacker tries a few passwords across many accounts to stay under a per user threshold. I would also repeat the experiment with many more users to see whether the thresholds hold up.